INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Department of Mining Engineering

Complete Mathematical Formulations:
Finite Element Method (FEM) & Distinct Element Method (DEM)

Course: Computational Methods and Mine Instrumentation

Prepared by:

Mobashshir Zainuddin

Supervised by:

Prof. Debashish Chakravarty

April 2026

PART 1: FINITE ELEMENT METHOD (FEM)

1.1 Introduction & Governing Philosophy

The Finite Element Method (FEM) is a powerful numerical technique for finding approximate solutions to boundary value problems (BVPs) described by partial differential equations (PDEs). The fundamental idea is to replace a continuous domain with a discretised representation — a **mesh** — composed of finite elements connected at nodes. The unknown field variable (e.g., displacement, temperature) is approximated within each element using simple polynomial **shape functions**. The global solution is assembled from all element contributions, yielding a system of algebraic equations solvable by standard linear algebra methods.

Core Philosophy of FEM:

- **Strong Form:** The PDE holds at every point in the domain (exact, often unsolvable analytically).
- **Weak Form:** The PDE is satisfied in an integral (weighted-residual) sense — the basis of FEM.
- **Galerkin Method:** Weight functions are chosen to be the same as the shape functions.
- **Principle of Minimum Potential Energy:** Equivalent variational approach for structural problems.

1.2 Strong Form: Governing Equation (1D Bar)

Consider a bar of length *L*, cross-sectional area *A(x)*, Young's modulus *E(x)*, and distributed body force *b(x)*. The axial equilibrium equation (the **strong form**) is the PDE that must be satisfied at every point in the domain:

```
d/dx [ A(x)·E(x) · du/dx ] + b(x) = 0      on x ∈ (0, L)
```

Subject to boundary conditions (BCs):

```
u(0) = 0                          [Essential / Dirichlet BC]
A·E · du/dx |_{x=L} = F           [Natural / Neumann BC]
```

The essential BC prescribes the displacement value (kinematic constraint), while the natural BC prescribes the traction (force). The strong form, while exact, requires C^2 continuity in the displacement field and is often impossible to solve analytically for complex geometries.

1.3 Weak Form (Weighted Residual / Galerkin Formulation)

To derive the weak form, we multiply the strong form PDE by a **test function** (or weight function) *w(x)* — chosen to satisfy the homogeneous essential BCs — and integrate over the entire domain [0, L]:

$$\int_0^L w \cdot \{ d/dx[A \cdot E \cdot du/dx] + b \} dx = 0$$

Applying **integration by parts** to the first term (transferring one derivative from *u* to *w*, reducing the continuity requirement from C^2 to C^1):

$$\int_0^L A \cdot E \cdot (dw/dx) \cdot (du/dx) dx = \int_0^L w \cdot b dx + w(L) \cdot F$$

This is the **weak (variational) form** of the problem. The boundary term $w(L) \cdot F$ appears naturally from integration by parts — this is why natural BCs are automatically incorporated into the weak form. The **Galerkin method** sets the test function $w = N$ (the same shape functions used to approximate *u*), which directly yields the stiffness equations.

1.4 Discretisation — Meshing the Domain

The bar of length *L* is divided into *n* elements. Each element *e* spans two adjacent nodes with element length:

$$l_e = L / n$$

Node coordinates are:

$$x_i = (i - 1) \cdot l_e , \quad i = 1, 2, ..., n+1$$

A mesh of *n* elements produces *n+1* nodes; each node has **1 degree of freedom (DOF)** — the axial displacement u_i. The connectivity array maps each element to its two node indices.

1.5 Shape Functions (Interpolation Functions)

Within element *e* spanning nodes *i* and $j = i+1$, the displacement field is approximated by a linear combination of **shape functions**. Using the local isoparametric coordinate:

$$\xi = (x - x_i) / l_e , \quad \xi \in [0, 1]$$

The two linear shape functions are:

$$N_1(\xi) = 1 - \xi \qquad (= 1 \text{ at node } i, \ = 0 \text{ at node } j)$$

$$N_2(\xi) = \xi \qquad (= 0 \text{ at node } i, \ = 1 \text{ at node } j)$$

Key properties: **Partition of unity**: $N_1 + N_2 = 1$; **Non-negativity**: $0 \leq N_k \leq 1$; Kronecker delta: $N_k(x_j) = \delta_{kj}$

The displacement within element *e* is approximated as:

$$u(x) \approx u^e(x) = N_1(x) \cdot u_i + N_2(x) \cdot u_j = [N]\{d^e\}$$

where $\{d^e\} = \{u_i, u_j\}^T$ is the element nodal displacement vector.

1.6 Strain–Displacement and Stress–Strain Relations

The axial strain ε is the first derivative of displacement. Differentiating the interpolation:

$$\varepsilon = du/dx = dN_1/dx \cdot u_i + dN_2/dx \cdot u_j = (1/l_e)[-1 \quad +1]\{d^e\} = [B]\{d^e\}$$

The **strain-displacement matrix [B]** for the 1D linear element is the 1×2 matrix:

$$[B] = (1/l_e)\ [-1, \quad +1]$$

[B] is constant over the element — strains are uniform within each linear element. Applying **Hooke's Law** (1D constitutive relation):

$$\sigma = E \cdot \varepsilon = E \cdot [B]\{d^e\}$$

1.7 Derivation of Element Stiffness Matrix

The element stiffness matrix is derived by applying the Galerkin condition to element *e*, or equivalently by **minimizing the element potential energy**

$$\Pi_e = (1/2) \int_{x_i}^{x_j} A \cdot E \cdot \varepsilon^2\, dx - \{d^e\}^T\{f^e\}$$

Setting $d\Pi_e/d\{d^e\} = 0$ and substituting $\varepsilon = [B]\{d^e\}$:

$$[k^e] = \int_{x_i}^{x_j} A \cdot E \cdot [B]^T[B]\, dx$$

For uniform *A* and *E*, $[B]^T[B]$ is constant over the element, so the integral evaluates trivially:

$$[k^e] = (AE/l_e) \cdot \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Local Element Stiffness Matrix $[k^e]$:

DOF	u_i	u_j
u_i	AE/l_e	$-AE/l_e$
u_j	$-AE/l_e$	AE/l_e

1.8 Global Stiffness Matrix Assembly

The global stiffness matrix [K] is assembled by superimposing all element contributions at their respective global DOF positions — the **Direct Stiffness Method**:

$$[K] = \Sigma_e\ [A_e]^T\ [k^e]\ [A_e] \qquad \text{(Boolean scatter matrix form)}$$

In Python: *K[np.ix_(element_dofs, element_dofs)] += k_local*

For a 5-element bar the 6×6 global matrix (where $k = AE/l_e$) takes the following **tridiagonal banded structure**:

DOF	0	1	2	3	4	5
0	k	−k	0	0	0	0
1	−k	2k	−k	0	0	0
2	0	−k	2k	−k	0	0
3	0	0	−k	2k	−k	0
4	0	0	0	−k	2k	−k
5	0	0	0	0	−k	k

Note the tridiagonal banded structure — a hallmark of 1D FEM. Interior nodes (1–4) carry stiffness *2k* (contributions from two adjacent elements). Boundary nodes (0, 5) carry stiffness *k* (one element only).

1.9 Application of Boundary Conditions

Essential (Dirichlet) BCs specify known displacement values. They are applied by the **elimination method**: rows and columns corresponding to constrained DOFs are removed from [K] and {F}:

```
System Before BC:    [K]{u} = {F}          size (n+1) × (n+1),
                                  singular

              After Eliminating Fixed DOF 0 (u₀ = 0):

[K_ff]{u_f} = {F_f}          size n × n,  non-singular, uniquely
                                  solvable
```

Subscript '*ff*' denotes the free-free partition of the partitioned global system. The matrix [K_ff] is symmetric, positive-definite, and can be solved uniquely.

1.10 Solution of the Global System

The reduced system is solved by **direct methods** (LU decomposition / Gaussian elimination) or iterative methods (conjugate gradient for large systems):

```
                    Nodal Displacement Solution:

                       {u_f} = [K_ff]⁻¹ {F_f}

                LU Decomposition:   [K_ff] = [L][U]

Forward solve:  [L]{y} = {F_f}    →   Back solve:  [U]{u_f} = {y}
```

Computational complexity: $O(n^3)$ for direct methods; $O(n)$ per iteration for iterative methods with sparse matrices.

Reaction Forces at fixed DOFs are recovered from:

```
{R} = [K_cf]{u_f}      (subscript 'c' = constrained, 'f' = free)
```

Equilibrium check: $\Sigma F_{ext} + \Sigma R = 0$ must hold.

1.11 Post-Processing: Secondary Variables

Once nodal displacements are known, secondary quantities are computed element-by-element:

$$\text{Axial Strain in Element e:} \quad \varepsilon^e = [B]\{d^e\} = (u_j - u_i) / l_e$$

$$\text{Axial Stress in Element e:} \quad \sigma^e = E \cdot \varepsilon^e = E(u_j - u_i) / l_e \quad [Pa]$$

$$\text{Axial Internal Force:} \quad P^e = A \cdot \sigma^e = AE(u_j - u_i) / l_e \quad [N]$$

Analytical Verification: For a uniform bar with end load F: $u(x) = Fx/(AE)$, $\varepsilon = F/(AE)$, $\sigma = F/A$. FEM with linear elements reproduces this exact solution.

PART 2: FEM IN 2D — CONSTANT STRAIN TRIANGLE (CST)

2.1 2D Elasticity: Governing Equations (Plane Stress)

In **plane stress** (thin plate, zero out-of-plane stress $\sigma_{zz} = 0$), the unknown fields are the 2D displacement vector $\mathbf{u}(x,y) = [u(x,y), v(x,y)]^T$. The governing equilibrium PDEs (Navier equations, **strong form**) in domain Ω are:

$$\partial\sigma_{xx}/\partial x + \partial\tau_{xy}/\partial y + b_x = 0$$

$$\partial\tau_{xy}/\partial x + \partial\sigma_{yy}/\partial y + b_y = 0$$

where b_x and b_y are body force components, σ denotes normal stresses, and τ denotes shear stress.

2.2 Strain–Displacement Relations (Small Deformation)

Under the assumption of small deformations (linear kinematics):

$$\varepsilon_{xx} = \partial u/\partial x \quad \text{(normal strain in x)}$$

$$\varepsilon_{yy} = \partial v/\partial y \quad \text{(normal strain in y)}$$

$$\gamma_{xy} = \partial u/\partial y + \partial v/\partial x \quad \text{(engineering shear strain)}$$

In vector form: $\{\varepsilon\} = \{\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}\}^T = [L]\{u\}$, where [L] is the differential operator.

2.3 Constitutive Law — D-Matrix (Plane Stress)

For an isotropic, linear elastic material under **plane stress** ($\sigma_{zz} = \tau_{xz} = \tau_{yz} = 0$), the stress-strain relationship is:

$$\{\sigma\} = [D]\{\varepsilon\}$$

The **constitutive matrix [D]** for plane stress is:

$$[D] = \frac{E}{(1-\nu^2)} \cdot \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}$$

[D] Components	ε_{xx}	ε_{yy}	γ_{xy}
σ_{xx}	$E/(1-\nu^2)$	$E\nu/(1-\nu^2)$	0
σ_{yy}	$E\nu/(1-\nu^2)$	$E/(1-\nu^2)$	0
τ_{xy}	0	0	$E / 2(1+\nu)$

Note: For **plane strain** ($\varepsilon_{zz} = 0$), the [D] matrix uses $(1-\nu)$ replacing 1 on the diagonal, and $\nu/(1-\nu)$ replacing ν.

2.4 CST Element: Shape Functions in 2D

The **Constant Strain Triangle (CST)** is the simplest 2D element: 3 nodes, 2 DOF per node (u, v), giving **6 DOF per element**. The displacement field is linear within the element, so strains are constant — hence the name.

Using **area coordinates (natural coordinates)**, the shape functions are:

$$N_k = (a_k + b_k \cdot x + c_k \cdot y) / (2A) \qquad \text{for } k = 1, 2, 3$$

where A is the element area and the geometry constants are:

$$b_1 = y_2 - y_3, \quad b_2 = y_3 - y_1, \quad b_3 = y_1 - y_2$$
$$c_1 = x_3 - x_2, \quad c_2 = x_1 - x_3, \quad c_3 = x_2 - x_1$$

Node numbering must be **counter-clockwise (CCW)** to ensure positive area:

$$2A = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1) \qquad [A > 0 \text{ for CCW}]$$

The displacement field within the element is:

$$u(x,y) = N_1 \cdot u_1 + N_2 \cdot u_2 + N_3 \cdot u_3$$
$$v(x,y) = N_1 \cdot v_1 + N_2 \cdot v_2 + N_3 \cdot v_3$$
$$\{d^e\} = \{u_1, v_1, u_2, v_2, u_3, v_3\}^T \quad \rightarrow \quad \text{6-DOF element vector}$$

2.5 Strain-Displacement Matrix [B] for CST

Differentiating the displacement interpolation gives strains. Since N_k are linear in x and y, all derivatives are constants — confirming constant strain within the element:

$$\{\varepsilon\} = [B]\{d^e\}$$

The **B-matrix [3×6]** for the CST element is:

$$[B] = (1/2A)\begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}$$

Rows correspond to {ε_xx, ε_yy, γ_xy}; columns correspond to $\{u_1, v_1, u_2, v_2, u_3, v_3\}$.

2.6 Element Stiffness Matrix for CST

Since [B] and [D] are **constant over the entire CST element**, the element stiffness matrix is:

$$[k^e] = \iint_{\{\Omega_e\}} [B]^T [D] [B] \cdot t \cdot dA = t \cdot A_e \cdot [B]^T [D] [B]$$

where t is the plate thickness and A_e is the element area. The integration is **trivial** because the integrand is constant — **no Gauss quadrature is required**. This is the key computational advantage of the CST element. The result is a 6×6 symmetric positive semi-definite matrix.

2.7 Assembly, BCs, and Solution (2D)

Assembly follows the same direct stiffness method as 1D. Each node contributes 2 DOFs (u, v), so the global DOF mapping for node i is:

$$u_i \rightarrow \text{global DOF } 2i$$
$$v_i \rightarrow \text{global DOF } 2i+1$$

The full 2D system is:

$$[K_{global}]\{U\} = \{F\} \qquad \text{size: } 2 \cdot N_{nodes} \times 2 \cdot N_{nodes}$$

Fixed-edge BCs: zero rows/columns for constrained DOFs → reduced system [K_ff]{U_f} = {F_f}.

2.8 Von Mises Stress (Failure Criterion)

For plane stress, the **Von Mises equivalent stress** is used to assess yielding. It combines the stress components into a scalar measure of distortional strain energy:

$$\sigma_{vm} = \sqrt{\sigma_{xx}^2 - \sigma_{xx} \cdot \sigma_{yy} + \sigma_{yy}^2 + 3\tau_{xy}^2}$$

Yielding occurs when $\sigma_{vm} \geq \sigma_{yield}$ (the material uniaxial yield strength). The Von Mises criterion is the most widely used for ductile metals.

PART 3: DISTINCT ELEMENT METHOD (DEM)

3.1 Introduction and Physical Basis

The **Distinct (or Discrete) Element Method (DEM)** treats a granular assembly as a collection of **rigid (or slightly deformable) particles** that interact through contact forces. Unlike FEM which models continua, DEM explicitly tracks each particle's **translation and rotation**. DEM is essential for modelling soils, rocks, powders, and other discontinuous media where failure occurs along particle boundaries.

DEM vs FEM — Key Conceptual Differences:

Attribute	FEM	DEM
Material	Continuum (Hooke's Law)	Discontinuous rigid particles
Unknowns	Nodal displacements {u}	Particle positions & velocities
Gov. Equation	[K]{u} = {F}	m·a = F (per particle)
Time Integration	Static or implicit Newmark	Explicit Euler/Verlet (small Δt)
Contact Model	Not applicable (shared nodes)	Linear Spring-Dashpot (LSD)
Best For	Structural deformation, stress analysis	Granular flow, fracture, rock mechanics

3.2 Equations of Motion — Newton's 2nd Law

Each particle p obeys Newton's laws for both translation and rotation. These are integrated explicitly at each time step.

Translational Motion:

$$m_p \cdot a_p = m_p \cdot d^2x_p/dt^2 = F_{p_total} = F_{p_gravity} + \Sigma_q F_{p_contact,q}$$

The sum is over all particles q currently in contact with particle p. Gravity: $F_{p_gravity} = m_p \cdot g$

Rotational Motion (2D):

$$I_p \cdot \alpha_p = I_p \cdot d^2\theta_p/dt^2 = \Sigma_q M_{p_contact,q}$$

where $I_p = (1/2)m_p r^2$ is the moment of inertia for a circular disc, and M is the torque from the tangential contact force times the moment arm.

3.3 Contact Detection

Contact between circular particles (2D) or spherical particles (3D) is detected geometrically. For two particles i and j with radii r_i and r_j:

$$\text{Centre-to-centre distance:} \quad d_{ij} = \|x_i - x_j\|$$

$$\text{Contact exists if:} \quad d_{ij} < r_i + r_j$$

The **normal overlap (penetration depth)** representing local elastic deformation at the contact point is:

$$\delta_n = (r_i + r_j) - d_{ij} \qquad [\delta_n > 0 \text{ in contact}, \ \delta_n \leq 0 \text{ no contact}]$$

The **unit normal vector** pointing from particle j to i is:

$$\hat{n}_{ij} = (x_i - x_j) / d_{ij}$$

3.4 Contact Force Model — Linear Spring-Dashpot (LSD)

The **Linear Spring-Dashpot (LSD) model** is the most widely used DEM contact law. It models each contact as a spring (elastic repulsion) in parallel with a dashpot (viscous damping).

3.4.1 Normal Force

$$F_n = k_n \cdot \delta_n - c_n \cdot v_{n_rel}$$

where k_n = normal spring stiffness [N/m]; c_n = normal damping coefficient [N·s/m]; and the normal relative velocity is:

$$v_{n_rel} = (v_i - v_j) \cdot \hat{n}_{ij}$$

Physical interpretation: $k_n \cdot \delta_n$ is the repulsive elastic force that prevents particle overlap. $c_n \cdot v_n$ is the dissipative damping force that converts kinetic energy to heat. **No tensile force:** F_n is clipped to zero if negative (particles cannot attract each other unless bonded).

3.4.2 Tangential Force (Sliding / Friction)

$$F_t = \min(k_t \cdot \delta_t - c_t \cdot v_{t_rel}, \ \mu \cdot F_n) \qquad [\text{Coulomb slip limit}]$$

where k_t = tangential stiffness, μ = friction coefficient, and δ_t = accumulated tangential displacement increment. The Coulomb limit ensures particles cannot sustain shear beyond the frictional capacity.

3.4.3 Wall Contacts

$$\text{Overlap with bottom wall (at } y = 0): \quad \delta_{wall} = r - y_n$$

$$\text{Wall normal force:} \quad F_{wall} = k_n \cdot \delta_{wall} - c_n \cdot v_y \quad (\text{if } \delta_{wall} > 0)$$

3.5 Critical Time Step — Stability Condition

DEM uses **explicit time integration**, which is **conditionally stable**. The time step Δt must satisfy:

```
Critical Time Step (undamped):   Δt_cr = 2 / ω_max = 2 · √(m / k□)
```

where ω_max = $\sqrt{(k_n/m)}$ is the natural frequency of the contact spring-mass system.

```
Recommended Practice:   Δt ≤ 0.1 · Δt_cr    (10% of critical)
```

Smaller Δt → more accurate but slower simulation. Damping slightly reduces ω_max. If Δt is too large, the simulation **diverges** (particle velocities grow unboundedly).

3.6 Time Integration Schemes

3.6.1 Euler-Cromer (Symplectic Euler)

The Euler-Cromer scheme updates velocity **before** position, giving it better energy conservation than standard forward Euler. It is 1st order accurate:

```
a□ = F(t□) / m                          [compute acceleration from
                                  forces]
v(t+Δt) = v(t) + a(t) · Δt              [velocity updated FIRST]
x(t+Δt) = x(t) + v(t+Δt) · Δt           [position uses NEW velocity]
```

3.6.2 Velocity Verlet Integration

The **Velocity Verlet** scheme is 2nd order accurate and is the standard choice for high-fidelity DEM. It requires one force evaluation per step:

```
x(t+Δt) = x(t) + v(t)·Δt + (1/2)·a(t)·Δt²     [update position]
a(t+Δt) = F(t+Δt) / m                         [compute new
                    forces at new positions]
v(t+Δt) = v(t) + (1/2)·[a(t) + a(t+Δt)]·Δt    [average
                          acceleration]
```

Velocity Verlet has excellent long-term energy conservation and is time-reversible (symplectic integrator).

3.7 Energy in DEM

Energy quantities are tracked to monitor simulation stability and physical correctness:

```
Kinetic Energy:          KE = Σ□ (1/2) m□ |v□|²          [J]
Gravitational PE:        PE = Σ□ m□ · g · y□             [J]
Elastic Strain Energy:   SE = Σ_{contacts} (1/2) k□ · δ□²  [J]
```

```
Total Mechanical Energy: TE = KE + PE + SE = constant  (no damping)
```

With damping: dTE/dt < 0 (energy dissipated). Monitoring TE provides the best diagnostic for numerical stability: **rising TE signals numerical instability** and requires reducing Δt or k_n.

PART 4: STEP-BY-STEP ALGORITHM SUMMARY

4.1 Complete FEM Algorithm (1D and 2D)

Step	Stage	What to Compute
1	**Preprocessing (Meshing)**	1D: linspace nodes, element connectivity. 2D: grid of nodes → 2 CST triangles per cell.
2	**Material Definition**	Assign E, ν (and A for 1D; t for 2D) to elements.
3	**Local Stiffness**	1D: $[k] = (AE/l_e)[[1,-1],[-1,1]]$. 2D: $[k] = t \cdot A \cdot [B]^T[D][B]$; compute b_k, c_k, area, B, D.
4	**Global Assembly**	K[dofs_e, dofs_e] += k_e for each element. 1D: 2 DOF/elem. 2D: 6 DOF/elem.
5	**Loads & BCs**	Build {F}. Essential BC: eliminate fixed rows/cols → [K_ff]{u_f} = {F_f}.
6	**Solve**	u_f = solve(K_ff, F_f). Recover reactions R = K_cf·u_f.
7	**Post-process**	1D: $\varepsilon = B \cdot d$, $\sigma = E\varepsilon$. 2D: $\{\varepsilon\} = [B]\{d^e\}$, $\{\sigma\} = [D]\{\varepsilon\}$, $\sigma_vm = \sqrt{(\sigma_xx^2 - \sigma_xx\sigma_yy + \sigma_yy^2 + 3\tau^2)}$.

4.2 Complete DEM Algorithm

Step	Stage	What to Compute
1	**Initialise**	Set positions x_p, velocities $v_p = 0$, accelerations $a_p = 0$. Assign r, m, k_n, c_n, k_t, μ, g.
2	**Force Reset**	$F_p = 0$ for all p. Then add gravity: $F_p \mathrel{+}= m_p \cdot g$.
3	**Contact Detection**	For all pairs (i,j): compute $d_\{ij\} = \|x_i - x_j\|$. If $d_\{ij\} < r_i + r_j$: contact detected → compute δ_n, $\hat{n}$.
4	**Contact Force**	Normal: $F_n = k_n \cdot \delta_n - c_n \cdot v_n$. Tangential: $F_t = k_t \cdot \delta_t$ (clipped by $\mu \cdot F_n$). Add $F_n \cdot \hat{n}$ to F_i; subtract from F_j.
5	**Wall Check**	For each wall: if particle overlaps wall → apply spring-dashpot repulsive force in wall-normal direction.
6	**Integrate**	a = F/m. Euler-Cromer: $v \mathrel{+}= a \cdot \Delta t$; $x \mathrel{+}= v \cdot \Delta t$. Or Velocity Verlet: $x \mathrel{+}= v \cdot \Delta t + 0.5a \cdot \Delta t^2$; recompute a; $v \mathrel{+}= 0.5(a_old + a_new) \cdot \Delta t$.

7	**Post-process**	Track KE, PE, TE. Record contact network, coordination number, force chains, packing fraction.

PART 5: MASTER EQUATION REFERENCE

5.1 FEM Equations Summary

Equation	Description	Domain
$[k] = (AE/l)[[1,-1],[-1,1]]$	1D bar element stiffness matrix	FEM 1D
$\varepsilon = B \cdot d = (u_j - u_i)/l_e$	1D axial strain (post-processing)	FEM 1D
$\sigma = E \cdot \varepsilon$	Hooke's Law (1D)	FEM 1D
$\{u_f\} = [K_ff]^{-1}\{F_f\}$	Nodal displacement solution	FEM 1D/2D
$[B] = (1/2A)[b\square, c\square \text{ terms}]$	CST strain-displacement matrix (3×6)	FEM 2D
$[D] = E/(1-\nu^2)[[1,\nu,0],...]$	Plane stress constitutive matrix	FEM 2D
$[k^e] = t \cdot A \cdot [B]^T[D][B]$	CST element stiffness (6×6)	FEM 2D
$\sigma_vm = \sqrt{(\sigma_xx^2 - \sigma_xx\sigma_yy + \sigma_yy^2 + 3\tau^2)}$	Von Mises equivalent stress	FEM 2D

5.2 DEM Equations Summary

Equation	Description	Domain
$\delta\square = (r_i + r_j) - d_\{ij\}$	Normal overlap / penetration depth	DEM
$F\square = k\square \cdot \delta\square - c\square \cdot v\square$	LSD normal contact force	DEM
$F\square = \min(k\square \cdot \delta\square, \mu \cdot F\square)$	Tangential force with Coulomb limit	DEM
$m \cdot a = F_gravity + \Sigma F_contact$	Newton's equation of motion per particle	DEM
$\Delta t_cr = 2\sqrt{(m/k\square)}$	Critical time step for stability	DEM
$v \mathrel{+}= a \cdot \Delta t; \quad x \mathrel{+}= v \cdot \Delta t$	Euler-Cromer integration (1st order)	DEM
$x \mathrel{+}= v \cdot \Delta t + 0.5a \cdot \Delta t^2; \ ...$	Velocity Verlet integration (2nd order)	DEM
$TE = KE + PE + SE = const.$	Energy conservation (no damping)	DEM

PART 6: PYTHON IMPLEMENTATION GUIDE

6.1 FEM 1D — Core Python Pattern

The following Python code implements the complete 1D FEM pipeline for a bar under axial load:

```
import numpy as np


# --- Parameters ---
L, n_el = 1.0, 5;  E, A = 210e9, 0.01
le = L / n_el;  n_nodes = n_el + 1
nodes = np.linspace(0, L, n_nodes)
elements = [[i, i+1] for i in range(n_el)]

# --- Local stiffness (Step 3) ---
k_loc = (A*E/le) * np.array([[1,-1],[-1,1]])

# --- Global assembly (Step 4) ---
K = np.zeros((n_nodes, n_nodes))
for el in elements:
    K[np.ix_(el, el)] += k_loc

# --- BCs and solve (Steps 5 & 6) ---
F = np.zeros(n_nodes);  F[-1] = 10000  # 10 kN end load
u = np.zeros(n_nodes)
u[1:] = np.linalg.solve(K[1:,1:], F[1:])

# --- Post-process (Step 7) ---
for i,j in elements:
    eps=(u[j]-u[i])/le; sig=E*eps; print(f'σ={sig:.4e} Pa')
```

6.2 FEM 2D CST — Core Python Pattern

```
def cst_stiffness(coords, E, nu, t):
    x1,y1=coords[0]; x2,y2=coords[1]; x3,y3=coords[2]
    area = 0.5*abs((x2-x1)*(y3-y1)-(x3-x1)*(y2-y1))
    b1=y2-y3; b2=y3-y1; b3=y1-y2
    c1=x3-x2; c2=x1-x3; c3=x2-x1
    B = (1/(2*area))*np.array([
        [b1,0,b2,0,b3,0],
        [0,c1,0,c2,0,c3],
        [c1,b1,c2,b2,c3,b3]])
    D = (E/(1-nu**2))*np.array([
        [1,nu,0],[nu,1,0],[0,0,(1-nu)/2]])
    return t*area*(B.T @ D @ B), B, area
```

6.3 DEM — Core Time-Step Loop (Euler-Cromer)

```
for step in range(num_steps):
    forces = np.zeros_like(pos)
    forces[:,1] += gravity * mass     # gravity
```

```
    for i in range(N):
        for j in range(i+1, N):
            diff = pos[i] - pos[j]
            dist = np.linalg.norm(diff)
            if dist < 2*radius:
                n_hat = diff / dist
                delta = 2*radius - dist
                v_rel_n = np.dot(vel[i]-vel[j], n_hat)
                Fn = kn*delta - cn*v_rel_n
                Fn = max(Fn, 0)          # no tension
                forces[i] += Fn * n_hat
                forces[j] -= Fn * n_hat
    # Euler-Cromer integration
    acc = forces / mass
    vel += acc * dt
    pos += vel * dt
```

6.4 Numerical Tips and Common Pitfalls

Method	Tip / Pitfall
FEM	Ensure CCW node numbering for 2D CST elements — area must be positive. Degenerate (flat) triangles cause singular [B].
FEM	Always verify equilibrium: sum of reactions + applied loads = 0. This is the single best check for assembly correctness.
FEM	For 1D uniform bar with linear elements, FEM gives the exact solution (linear displacement is in the polynomial space).
FEM	CST is overly stiff (locking) — use more elements or higher-order elements (LST, Q4/Q8) for bending-dominated problems.
DEM	Choose $\Delta t < 0.1 \cdot \sqrt{(m/k_n)}$. Too large → particles explode (numerical blow-up); too small → simulation is excessively slow.
DEM	Use grid-based spatial hashing for contact detection when $N > 500$. This reduces $O(N^2)$ contact search to $O(N)$.
DEM	Clip contact forces to $F_n \geq 0$: particles cannot pull each other unless explicitly bonded (parallel-bond model).
DEM	Monitor total energy TE each step: rising TE indicates numerical instability — reduce Δt or k_n immediately.